Queuing theory: Why 80–90%?

Imagine you are the manager of a supermarket. What should be the goal for daily utilization of your cashiers? It's tempting to go for a high number, maybe 90%. But it turns out that the wait time for your customers is very high when your cashiers are used for 90% of the day, because customers don't arrive at the same time at the queue. The theory behind this is called *queuing theory*, and it turns out that wait time is exponential to the utilization of a resource. This applies not only to cashiers but also to network cards, CPU, hard disks, and so on. Keep in mind that this sidebar simplifies the theory and assumes an M/D/1 queuing system: Markovian arrivals (exponentially distributed arrival times), deterministic service times (fixed), one service center. To learn more about queuing theory applied to computer systems, we recommend *Systems Performance: Enterprise and the Cloud* by Brendan Gregg (Prentice Hall, 2013) to get started.

When you go from 0% utilization to 60%, wait time doubles. When you go to 80%, wait time has tripled. When you to 90%, wait time is six times higher, and so on. If your wait time is 100 ms during 0% utilization, you already have 300 ms wait time during 80% utilization, which is already slow for an e-commerce website.

You might set up an alarm to trigger if the 10-minute average of the `EngineCPUUtilization` metric is higher than 80% for one out of one data points, and if the 10-minute average of the `SwapUsage` metric is higher than 67108864 (64 MB) for one out of one data points. These numbers are just a rule of thumb. You should load-test your system to verify that the thresholds are high or low enough to trigger the alarm before application performance suffers.

11.5.2 Is my memory sufficient?

The `Evictions` metric is reported by Memcached and Redis. If the cache is running out of memory and you try to add a new key-value pair, an old key-value pair needs to be deleted first. This is called an eviction. By default, Redis deletes the least recently used key, but only for keys that define a TTL. This strategy is called `volatile-lru`. On top of that, the following eviction strategies are available:

- `allkeys-lru`—Removes the least recently used key among all keys
- `volatile-random`—Removes a random key among keys with TTL
- `allkeys-random`—Removes a random key among all keys
- `volatile-ttl`—Removes the key with the shortest TTL
- `noeviction`—Does not evict any key